VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI

REPORT ON

BIG DATA ANALYTICS [BCS714D]

Distributed Food and Nutrient Analytics Using MongoDB, Hadoop, Pig and Hive

Track B — Core Themes and Practical Dashboard Extension

Submitted by:

No	Student name	USN	Student signature	Marks awarded	Faculty signature
1	Abhishek R	1JS23CS004			
2	Adithya V	1JS23CS005			
3	Dhanush Singh G	1JS23CS048			

Guided by: Sandhiya B

JSS ACADEMY OF TECHNICAL EDUCATION, Bengaluru – 560060

Department of Computer Science and Engineering

Academic year: 2026–2027

ABSTRACT

This project implements a reproducible food and nutrient analytics workflow using the USDA FoodData Central CSV release dated October 31, 2024. It addresses the three core themes of the BCS714D Big Data Analytics assignment and the Track B practical extension. MongoDB stores the source tables and supports linked document aggregation and index profiling. Hadoop provides HDFS storage and YARN execution of Python Streaming MapReduce. Pig transforms the prepared records, while Hive executes identical queries over Text and gzip RCFile tables. A Flask dashboard presents analytical evidence and controls predefined laptop workflows.

The complete source package was prepared on the project laptop and all 34 CSV tables were imported into MongoDB. The preparation manifest contains 2,063,876 foods, 477 nutrient definitions and 26,848,323 nutrient observations with zero preparation rejects. End-to-end development checks passed for linked subsets of 10,000 and 50,000 foods; the latter contains 1,126,928 linked nutrient observations. In the recorded 10,000-food point-query experiment, an index examined one document and one key instead of scanning 10,000 documents, while returning the same row.

Source fingerprints, input accounting and independent reconciliation establish consistent group counts, totals and means across the analytical systems. The reported experiments use development subsets in the laptop container environment. The dashboard presents the selected results, benchmark artifacts and live execution progress through a single interface. Together, the processing pipeline and dashboard implement the core analytical themes and the Track B practical extension.

Keywords: FoodData Central, MongoDB, HDFS, YARN, MapReduce, Pig, Hive, RCFile, Flask.

CONTENTS

Abstract....2

CHAPTER 1: INTRODUCTION....5

CHAPTER 2: BACKGROUND AND REQUIREMENT ANALYSIS....7

CHAPTER 3: DATASET AND EXPERIMENTAL SCOPE....9

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE....12

CHAPTER 5: PART 1 — IMPLEMENTATION OF THE CORE THEMES....15

CHAPTER 6: PART 2 — FLASK DASHBOARD AND JOB CONTROLS....19

CHAPTER 7: TESTING, RESULTS AND DISCUSSION....21

CHAPTER 8: CONCLUSION....24

REFERENCES....25

LIST OF FIGURES AND TABLES

Figure 3.1 — Core food and nutrient relationships.....10

Figure 4.1 — Data processing and reconciliation workflow.....12

Figure 4.2 — Three-system deployment architecture.....13

Figure 7.1 — Documents examined in the recorded point-query comparison.....22

Table 2.1 — Assignment requirements and implemented components.....8

Table 3.1 — Full-release data inventory.....9

Table 3.2 — Experimental datasets.....10

Table 4.1 — Software and key configuration.....14

Table 5.1 — Required MongoDB stages.....15

Table 7.1 — Recorded 10,000-food profiling observations.....21

Table 7.2 — Completed functional validation.....23

CHAPTER 1: INTRODUCTION

1.1 Project background

Food composition data supports nutritional research, product comparison and public information services. USDA FoodData Central distributes food descriptions, nutrient definitions and nutrient observations in separate relational CSV tables. A food can have several nutrient observations, while a nutrient definition supplies its name and measurement unit. The October 31, 2024 package therefore requires linked processing rather than treating each CSV as an unrelated spreadsheet [1, 2].

This project implements a reproducible analytics workflow for the BCS714D Big Data Analytics assignment under Track B. Part 1 covers MongoDB processing, distributed Hadoop MapReduce, and Pig/Hive analytics. Part 2 implements a Flask dashboard for viewing evidence and controlling predefined laptop workflows. The central engineering objective is to obtain consistent analytical answers across the tools while preserving the identity and scope of the input data.

1.2 Problem statement

Develop an integrated system for FoodData Central that stores the assigned release, connects food and nutrient records, performs the three required analytical themes and displays consistent results through a web interface. The design uses a Hadoop master and two workers for distributed storage and processing, with MongoDB for document analytics and Hive for storage-format comparison.

1.3 Objectives

The objectives are to preserve all source CSV tables in MongoDB; construct linked food documents and analytical pipelines; compare indexed and collection-scan queries; demonstrate HDFS file operations; execute Python Streaming jobs on YARN; clean records with distributed Pig; compare Hive Text and RCFile tables; and expose traceable results through Flask.

Reproducibility is supported by source fingerprints, input counts, configuration context, logs and result comparisons. Each development subset includes the selected foods and their linked nutrient observations so that every analytical system uses the same records.

1.4 Scope and deliverables

The primary output is a summary grouped by food type and nutrient identifier: observation count, total recorded amount and mean recorded amount. Protein is one supported nutrient among the complete nutrient definitions. This grouped view describes food categories and nutrient observations rather than individual named products.

The full release was prepared and all 34 CSV tables were imported into MongoDB. The experiments presented in this report use linked subsets of 10,000 and 50,000 foods on the laptop's three-container Hadoop environment. The system also includes a native three-system deployment configuration and a Flask dashboard as the Track B practical extension.

CHAPTER 2: BACKGROUND AND REQUIREMENT ANALYSIS

2.1 Technical background

MongoDB stores document collections and supports aggregation stages for filtering, grouping, projection, array expansion and multiple analytical branches. Indexes improve selected access patterns, but their benefit depends on the filter, data distribution and query plan. The executionStats output provides evidence about examined documents and keys rather than relying only on elapsed time [3].

Hadoop separates distributed storage from resource scheduling. HDFS maintains a file namespace and stores blocks on DataNodes; YARN schedules application containers on NodeManagers. Hadoop Streaming allows Python mapper and reducer programs to participate in MapReduce without implementing the full processing logic in Java [4, 5].

Pig expresses data transformations as a sequence of loads, filters and stores. Hive provides SQL-like analytical queries over files. RCFile uses a row-group and column-oriented organization; its behavior depends on workload, compression, table size and execution overhead. This project uses Pig 0.17.0 and Hive 2.3.9 for compatibility with the assigned legacy technologies [6, 7].

2.2 Existing approach and proposed approach

A spreadsheet or a single script can answer small nutrition queries, but it does not demonstrate the required distributed execution, heterogeneous data management or storage-format benchmarking. Independently written calculations also risk comparing different selected records, filtering rules or units.

The proposed approach uses the same linked records and cleaning semantics across analytical systems, then reconciles every analytical group against an independent calculation. It retains a complete raw import separately from the development subset and derived analytical documents. A dashboard presents measurements and their evidence scope rather than substituting example results.

2.3 Functional requirements

Table 2.1 — Assignment requirements and implemented components

Requirement	Implementation	Evidence retained
Document ingestion	Import every CSV into raw MongoDB collections	Import manifest and collection counts
Aggregation and indexing	Required stages, linked documents and indexes	Pipeline snapshots and executionStats
HDFS file management	mkdir, put, get, cat and rm -r	Command logs and byte-checked probe
Distributed MapReduce	Python mapper/reducer through YARN	Job IDs, output and success checks
Pig transformation	Distributed filtering of prepared TSV	Script and cleaned row counts
Hive comparison	Identical queries on Text and RCFile	Answers, timings and storage metrics
Track B extension	Flask results and laptop job controls	Browser checks, states and logs

2.4 Non-functional requirements

Correctness requires matching analytical groups and observation counts, with controlled tolerance for floating-point totals and means. Source fingerprints and run metadata establish traceability. Separate ingestion, preparation, processing and presentation modules support maintainability. Disk-backed preparation and bounded batch insertion limit memory use.

The laptop workflow switches between MongoDB and Hadoop services to manage available RAM. Dedicated output directories preserve results from separate experiments. The native deployment uses the physical Hadoop workers and a view-only dashboard for presenting their analytical outputs.

CHAPTER 3: DATASET AND EXPERIMENTAL SCOPE

3.1 Dataset provenance

The input is the official USDA FoodData Central CSV release dated October 31, 2024 [1, 2]. The downloaded archive is approximately 456 MB. Its SHA-256 fingerprint is recorded in preparation and import metadata, while ZIP verification checks the archive contents.

Recorded source SHA-256:

```
85b6268117a8f6b8032d67a06ccd51ed1287a53ebaaeb0b3e262165e2820cb9f
```

Table 3.1 — Full-release data inventory

Item	Recorded count	Description
CSV tables	34	Complete raw table inventory
Food records	2,063,876	Rows in the food table
Nutrient definitions	477	Nutrient names and units
Food-nutrient records	26,848,323	Linked observation source rows
Prepared nutrient rows	26,848,323	Successfully written during preparation

3.2 Core relational structure

The food table identifies a food by fdc_id and supplies its type and description. The food_nutrient table associates a food identifier with a nutrient identifier and a recorded amount. The nutrient table supplies the nutrient name and unit. Additional release tables retain supporting product, sample, method and classification information even when they do not participate in the primary nutrient summary.

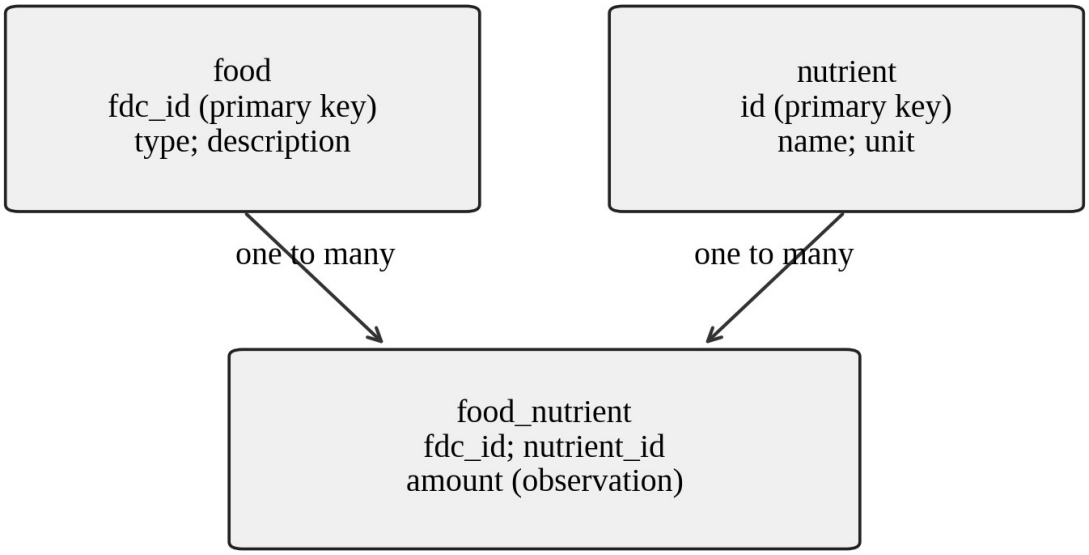


Figure 3.1 — Core food and nutrient relationships

The preparation layer joins these three core tables into a seven-column record containing food ID, food type, category, nutrient ID, nutrient name, unit and amount. This normalized TSV is suitable for line-based Hadoop input. Category values are preserved as nullable text because the source can contain category identifiers or labels.

3.3 Development subsets

Each development subset selects foods in source-row order and includes all their linked nutrient observations, together with the nutrient definitions. Consequently, a 50,000-food selection contains more than 50,000 total records. This deterministic selection provides a repeatable development workload.

Table 3.2 — Experimental datasets

Dataset	Food count	Description
Full release	2,063,876	Complete preparation and raw MongoDB import
Development subset 1	10,000	Linked MongoDB and distributed analytics
Development subset 2	50,000	Includes 1,126,928 linked nutrient records

The 10,000-food selection contains 9,994 branded foods and six experimental foods. The reported summaries describe the selected records. Source-order selection is used for reproducibility rather than population-representative nutritional estimation.

3.4 Meaning of analytical amounts

For a fixed food-type and nutrient group, the observation count is the number of accepted recorded measurements. Total amount is their sum, and mean amount is that sum divided by the count. Missing observations are not automatically replaced with zero. A stored zero is a valid value and remains in the calculation.

For example, three illustrative protein observations of 2 g, 4 g and 9 g produce a total of 15 g and a mean of 5 g. These illustrative values are not measured USDA project output. The calculation summarizes recorded observations; it is not the nutrition of a single named food, an amount consumed by a person, or a population-weighted dietary statistic. Comparison also requires attention to the source measurement basis; aggregation does not itself harmonize all serving or reporting contexts.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall workflow

The architecture separates raw storage, disk-backed preparation, distributed analytical execution and evidence presentation. MongoDB preserves original source strings and supplies document analytics. Python and SQLite prepare linked records without keeping the complete relational input in memory. HDFS stores the prepared files, MapReduce calculates summaries, Pig cleans records, and Hive compares storage/query alternatives. An independent reference scan checks the analytical outputs before the report or dashboard presents completion.

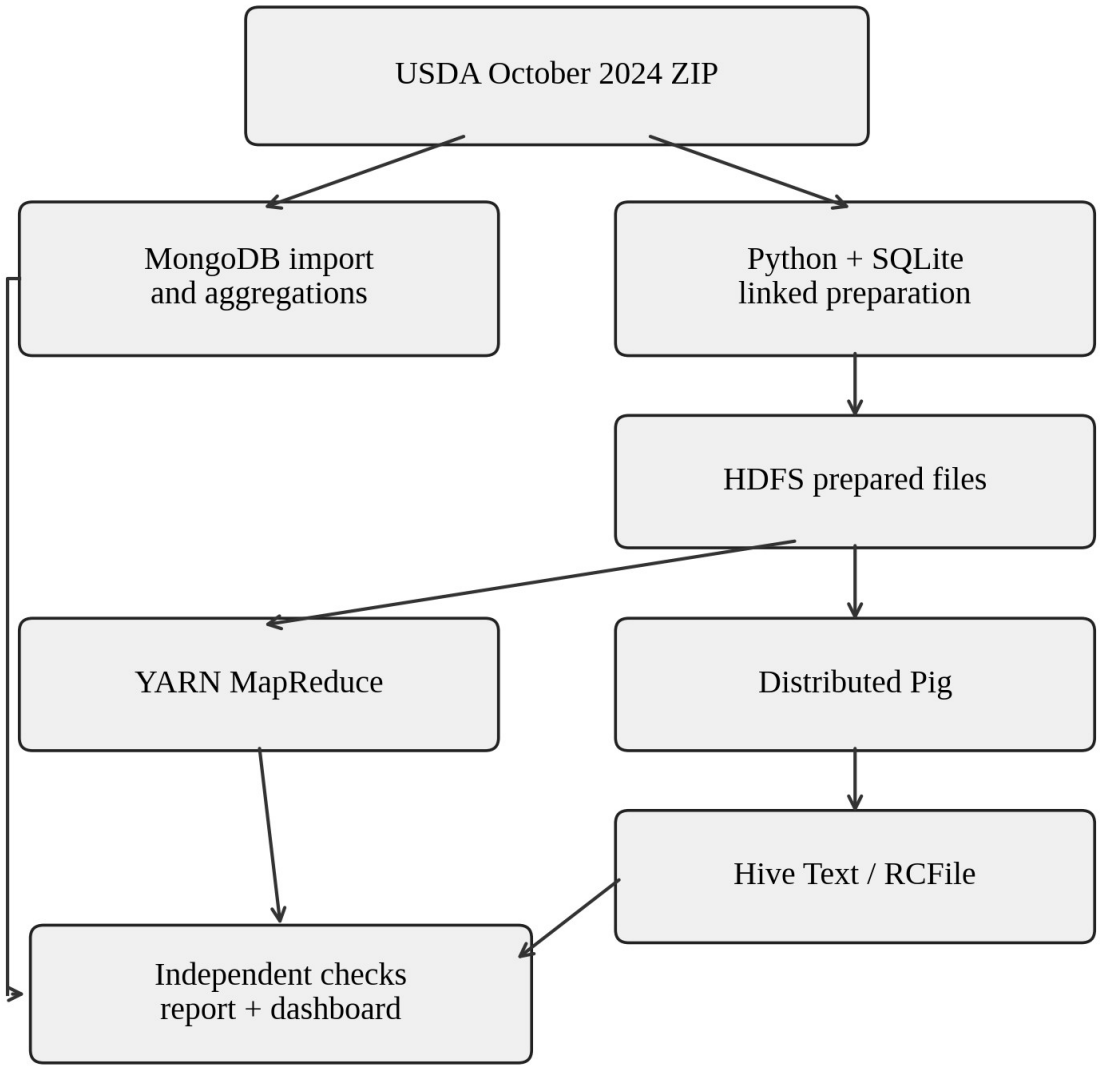


Figure 4.1 — Data processing and reconciliation workflow

The source SHA-256 connects imported data with prepared artifacts. Development exports additionally identify the selected records and parent release. The reconciliation layer checks this shared identity when comparing analytical outputs.

4.2 Physical and container deployment

The deployment has one MASTER and two workers. MASTER hosts the NameNode, ResourceManager and project clients, together with MongoDB, HiveServer2, Pig and Flask. WORKER1 and WORKER2 run DataNodes and NodeManagers, storing HDFS blocks and executing scheduled tasks.

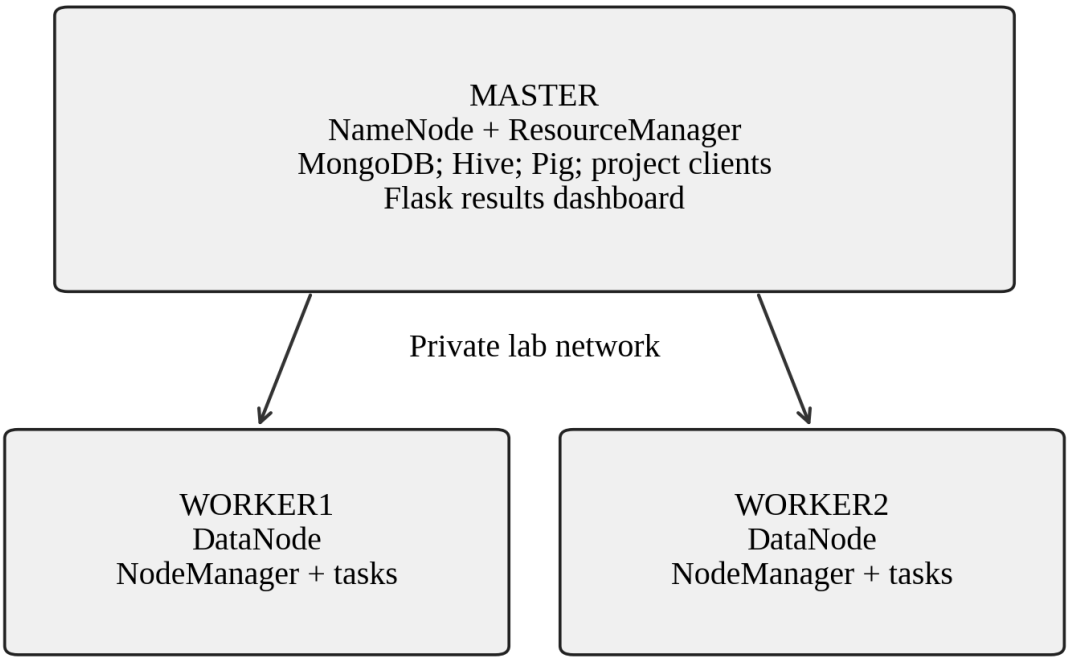


Figure 4.2 — Three-system deployment architecture

On the laptop, three Docker containers implement these Hadoop roles and share the host's processor, RAM and disk. The native lab configuration assigns the same roles to three separate Ubuntu computers. MongoDB remains a service on the master, and SQLite is a local disk-backed preparation helper.

4.3 Software and resource configuration

Table 4.1 — Software and key configuration

Component	Version / setting	Role
Operating system	Ubuntu 24.04 / WSL2 for laptop	Execution environment
Java	Java 8	Legacy Hadoop/Pig/Hive compatibility
Hadoop	2.10.2	HDFS, YARN and Streaming
Pig / Hive	0.17.0 / 2.3.9	Transformation and analytical SQL
MongoDB	7.0.16	Raw and derived document collections
Python libraries	PyMongo 4.10.1; Flask 3.1.2	Database access and web interface
HDFS replication	2	Two block replicas where workers permit
YARN per worker	2,560 MiB; 2 vcores	Configured task resources

The laptop has approximately 15.3 GB installed RAM, with 9.7 GiB available to WSL. Compose limits each Hadoop container to 4 GiB. The UI switches MongoDB and Hadoop services to manage memory.

4.4 Data and run-state design

Raw collections preserve source-row identifiers and CSV fields. Metadata records release identity, counts and completion. Derived food documents hold linked nutrient arrays. Unique run directories retain results, job states and atomic progress snapshots, while a process lock serializes UI execution.

CHAPTER 5: PART 1 — IMPLEMENTATION OF THE CORE THEMES

5.1 Theme 1: complete document ingestion

The ingestion program iterates over all CSV members of the selected archive and creates a raw_<table> collection for each table. It inserts records in bounded batches, preserving CSV strings and source-row identity. After each table, it compares the inserted count with a database count and updates the import manifest. The manifest becomes complete only after the table inventory has been processed.

Each import uses a dedicated database and records its table inventory. The laptop source database is fdc_home202410, while development subsets use separate generated databases. The native Part 1 driver provides the complete import-to-report workflow.

5.2 Linking and indexing

The analytical preparation creates an index on raw_food_nutrient.fdc_id and an index on raw_nutrient.id. A nested lookup associates each food with its nutrient rows and confirms a known nutrient definition. Numeric conversion admits finite amounts and valid nutrient IDs; the derived foods collection contains the linked observations. The derived food count is checked against the input manifest.

An additional index on raw_food.fdc_id supports the profiling query. Index definitions are retained with the result artifacts, connecting the recorded query plan to the indexes used by that experiment.

5.3 Aggregation pipelines

Table 5.1 — Required MongoDB stages

Stage	Project usage	Analytical purpose
$match	Restrict valid types and nonnegative observations	Consistent filtering
$group	Group by food type and nutrient	Counts, totals and means
$project	Select or calculate output fields	Compact analytical records

Stage	Project usage	Analytical purpose
$unwind	Expand the derived nutrient array	One observation per pipeline row
$facet	Food count, type summary and nutrient coverage	Multiple summaries in one pipeline

The food-type pipeline counts foods by data_type. The nutrient pipeline expands nutrient arrays, filters accepted amounts, groups by food type and nutrient ID, and calculates count, sum and average. The overview pipeline uses a facet to return several summaries. The pipeline definitions are saved with the results to show what actually ran.

5.4 Query performance profiling

The same food-ID equality filter is executed through two forced plans: a collection scan using the natural-order hint and an indexed query using idx_fdc_id. The program alternates plan order across five repetitions per plan. It records returned rows, examined documents, examined keys, server execution time and client wall time. PyMongo's explain command with executionStats verbosity is equivalent to the required MongoDB shell profiling operation.

Repeated measurements describe warm-cache development behavior. A recorded server time of 0 milliseconds indicates a duration below the timer's resolution; document and key counts provide an additional measure of query work.

5.5 Theme 2: HDFS file operations

The HDFS demonstration creates dedicated run directories, uploads the original or exported input and prepared files, reads a probe, downloads it and compares the bytes. It deletes only the run's temporary demonstration directory. The raw/prepared data and analytical results remain available for later stages and evidence inspection.

The commands cover hdfs dfs -mkdir, -put, -get, -cat and -rm -r. The byte comparison verifies retrieval integrity. HDFS reports, worker registration and a sample YARN application establish cluster readiness.

5.6 Distributed MapReduce algorithm

The mapper reads the seven-column prepared TSV and validates its field count, nutrient ID and finite amount. Negative amounts are skipped and counted through a Hadoop counter.

For each retained record, it emits a composite key consisting of food type and nutrient ID, together with the amount and a count of one.

The framework partitions and sorts intermediate records by key. Each reducer processes one key group, adds observation counts and calculates a numerically stable sum using math.fsum. Mean amount is total amount divided by observations. The implementation does not compute an unweighted mean of mapper means, which would be incorrect for groups of unequal size.

Conceptually, for accepted amounts x1 through xn in group g: count(g) = n; total(g) = sum(xi); mean(g) = total(g) / n. The Streaming launcher specifies YARN as the execution framework, distributes mapper.py and reducer.py through the file cache, requests two reducers by default and checks the HDFS _SUCCESS marker.

The run retains YARN application identifiers, MapReduce output and task diagnostics so that execution can be traced from submission to the resulting HDFS files.

5.7 Theme 3: Pig transformation

Pig loads the prepared TSV using the explicit seven-column schema. The filter requires a food ID, nutrient ID, food type and amount, and excludes negative amounts. It stores the cleaned observations in a dedicated HDFS output location. The project launches Pig in MapReduce mode rather than local mode.

Preparation and cleaning have different responsibilities. Preparation resolves references and rejects malformed or nonfinite amounts with explicit rejection records. Pig applies the analytical filter to the already normalized structured input. This separation avoids using a naive line-based CSV split on quoted source descriptions containing commas, tabs or newlines.

5.8 Hive Text and RCFile storage

The Hive layer creates an external Text table pointing at the Pig output and an external RCFile table with the same columns. The RCFile table is populated from the Text table, with gzip output compression enabled. Both tables therefore represent the same cleaned observations. Dedicated external locations retain the preceding HDFS data.

The first query summarizes every food-type/nutrient/unit group. The second restricts the records to nutrient 1003 with unit g, providing a protein-focused projection of the same analytical design. Local automatic execution is disabled and the execution engine is set to

MapReduce. The queries must return equivalent groups, counts, totals and means before performance is compared.

5.9 Benchmark method and reconciliation

For each query/table pair, the benchmark executes one warmup and then the configured number of measured repetitions in a seeded randomized order. With two queries, two formats and five measured repetitions, the full benchmark contains four warmups and twenty measured executions. The short development comparison uses one measured repetition per pair, or eight executions in total.

Elapsed time includes Beeline client startup; it is not pure storage scan time. The program records individual measurements and summarizes measured runs separately from warmups. It obtains logical bytes and replication-adjusted space from Hadoop's ContentSummary interface. Comparing Text with gzip RCFile measures both layout and compression choices, so the result is not a controlled isolation of file format alone.

An independent reference calculation scans prepared records with memory proportional to the number of analytical groups. It checks consistent units for each nutrient ID and uses compensated accumulation. MongoDB, MapReduce and Hive outputs must match exact group membership and counts. Totals and means use relative and absolute tolerances of 1e-9. Source hashes, preparation accounting and Pig output counts must also reconcile.

CHAPTER 6: PART 2 — FLASK DASHBOARD AND JOB CONTROLS

6.1 Purpose and information layout

The Flask application is the Track B practical extension. It separates View results from Run analysis and connects each assignment theme with its implementation and recorded output. The result view identifies the selected dataset, source fingerprint and analytical scope.

The dashboard displays nutrient summaries, MongoDB index comparisons, Hive query timings and storage measurements. Its theme sections explain collection organization, aggregation operators, HDFS commands, mapper/reducer programs, Pig transformation and Hive queries.

6.2 Laptop execution workflow

The interface offers standard subset sizes, a custom size from one to 100,000 foods, and an entire-dataset option. The source database is fixed. A subset workflow copies selected foods with linked nutrient records, runs MongoDB analytics, exports the same records and launches the Hadoop/Pig/Hive checks.

Service switching is automatic: MongoDB runs during document analytics, followed by Hadoop for the distributed stages. The full-data workflow verifies the original archive against the existing import, prepares the input and runs the complete analytical pipeline without duplicating the raw import.

6.3 Current progress and saved history

The analysis page shows Ready when no job is active. Previous results are selected from history, keeping saved measurements separate from current execution. A running job is followed automatically, and its completed results can be opened directly.

Progress refreshes every two seconds and includes the active stage, operation description, elapsed time, process identifier and latest output. Hive progress identifies the query, table format and warmup or measured execution. Technical logs provide command-level details alongside the plain-language progress display.

6.4 Background job management

A detached worker executes the predefined workflow independently of the browser. A process lock permits one UI job at a time, and persisted state allows reconnection after a

dashboard restart. The worker records stage completion, analytical outputs and diagnostics under a unique job directory.

Local control requests use validated actions, dataset sizes, origin checks and request tokens. The implementation invokes fixed command lists. This keeps the interface focused on the project's analytical workflow.

6.5 Dashboard behavior on three physical systems

In the native deployment, MASTER launches the Part 1 workflow with cluster mode client. The dashboard runs in view-only mode and displays the saved analytical results. The same processing modules and result schema support both the container environment and the three-system configuration.

CHAPTER 7: TESTING, RESULTS AND DISCUSSION

7.1 Validation approach

Validation combines recorded USDA subset experiments, automated implementation tests and browser checks. The implementation validation record contains 46 passing Python tests, including MongoDB integration. Browser checks cover result viewing, history selection, job submission, live progress, reconnection and mobile layout.

7.2 Full preparation and raw ingestion outcome

The complete source package was prepared and all 34 CSV tables were imported into MongoDB. The recorded counts include 2,063,876 foods, 477 nutrient definitions and 26,848,323 nutrient observations. The preparation manifest and import metadata provide consistent counts and a matching source fingerprint.

7.3 Ten-thousand-food index experiment

Table 7.1 — Recorded 10,000-food profiling observations

Metric	Forced collection scan	Indexed query
Returned documents	1 per execution	1 per execution
Documents examined	10,000 per execution	1 per execution
Index keys examined	0	1 per execution
Recorded times, milliseconds	8, 8, 6, 12, 23	1, 0, 0, 0, 0
Median server time	8 milliseconds	0 milliseconds at timer resolution

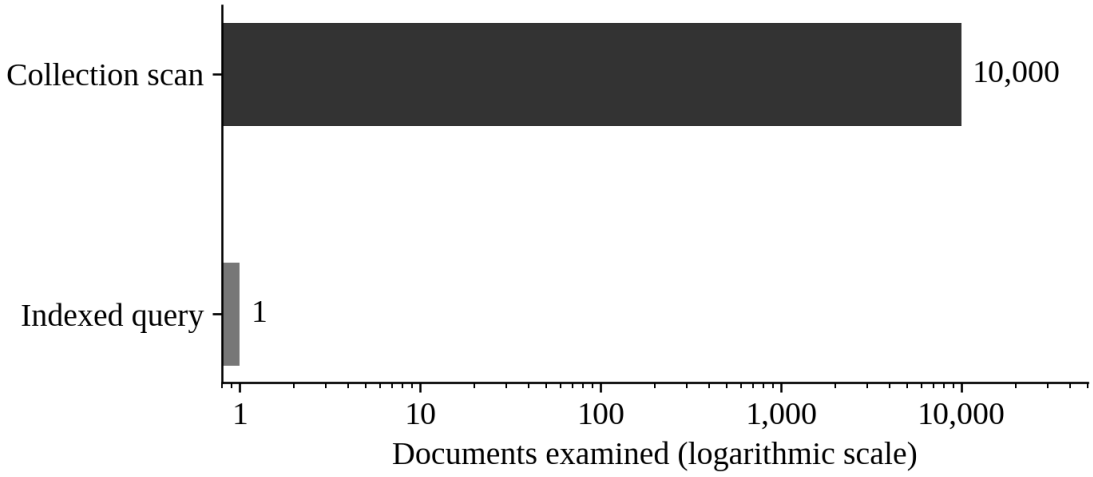


Figure 7.1 — Documents examined in the recorded point-query comparison

The query plans returned the same food while examining very different numbers of records. The indexed query reduced examined documents by 99.99 percent for this particular point lookup. This is a work-reduction observation, not a claim that every query is 10,000 times faster. The zero-millisecond measurements cannot support a finite time-speedup calculation without higher-resolution evidence.

7.4 Fifty-thousand-food distributed outcome

The 50,000-food experiment contains 1,126,928 linked nutrient observations and 477 nutrient definitions. MongoDB analytics completed with ten profiling executions. The exported records then passed the distributed integration checker against the saved MongoDB summaries and independent prepared-data calculations.

HDFS/YARN verification, MapReduce, Pig transformation and Hive Text/RCFile query comparison completed successfully for this selected workload. The retained evidence directory is data/realdev_1792244956898668695, containing the run marker, distributed log, reducer output and Hive artifacts.

7.5 Hive and cross-system result interpretation

The development runs verified that Text and RCFile queries return equivalent analytical groups, counts, totals and means. MongoDB and MapReduce summaries also matched the independent reference calculation for the selected records. This establishes consistent filtering and aggregation across the systems.

The benchmark records elapsed times for each query and storage sizes for each table. Warmups are excluded from measured-run summaries. Timings include Beeline startup, and

the RCFile experiment uses gzip compression; these settings define the interpretation of the comparison.

7.6 Functional validation matrix

Table 7.2 — Completed functional validation

Component	Validation	Outcome
Full raw preparation/import	Source counts and manifest identity	Completed
MongoDB subset analytics	10,000 and 50,000-food runs	Passed
HDFS operations	Upload/read/download and byte comparison	Passed
MapReduce / Pig	Independent summaries and cleaned counts	Passed
Hive Text / RCFile	Equivalent query results	Passed
Dashboard	Automated and browser checks	Passed

CHAPTER 8: CONCLUSION

The project integrates MongoDB document processing, HDFS storage, YARN MapReduce, Pig transformation and Hive analytics for USDA FoodData Central. Its three core themes cover ingestion and profiling, distributed file and data processing, and comparison of Text and RCFile analytical storage. The Flask dashboard provides a unified interface for the Track B practical extension.

The complete release was prepared and imported into MongoDB. The reported analytical experiments use linked subsets of 10,000 and 50,000 foods, including 1,126,928 nutrient observations in the larger subset. Cross-system reconciliation confirms consistent counts, totals and means for those selected workloads. The recorded point-query experiment also demonstrates the reduction in examined documents provided by an appropriate index.

The resulting system connects source data, processing stages and analytical outputs through a reproducible workflow. Its architecture and dashboard make the role of each technology visible while retaining a common basis for comparing the results.

REFERENCES

[1] USDA Agricultural Research Service, “FoodData Central: Download Datasets,” https://fdc.nal.usda.gov/download-datasets/ . Dataset information for the October 2024 release; reference recorded October 21, 2026.

[2] USDA Agricultural Research Service, “FoodData Central CSV, October 31, 2024,” https://fdc.nal.usda.gov/fdc-datasets/FoodData_Central_csv_2024-10-31.zip . Official source archive URL.

[3] MongoDB, “Explain Results and Aggregation Pipeline,” https://www.mongodb.com/docs/manual/reference/explain-results/ and https://www.mongodb.com/docs/manual/core/aggregation-pipeline/ . Reference for aggregation and executionStats concepts.

[4] Apache Software Foundation, “Hadoop Cluster Setup,” https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html . Faculty-supplied cluster setup reference.

[5] Apache Software Foundation, “Hadoop Streaming,” https://hadoop.apache.org/docs/r2.10.2/hadoop-streaming/HadoopStreaming.html . Reference for Streaming command execution.

[6] Apache Software Foundation, “Apache Pig 0.17 Documentation,” https://pig.apache.org/docs/r0.17.0/ . Reference for Pig Latin and distributed execution.

[7] Apache Software Foundation, “Apache Hive Language Manual,” https://hive.apache.org/docs/latest/language/languagemanual/ . General Hive language reference; implementation uses Hive 2.3.9.

[8] Pallets Projects, “Flask Documentation,” https://flask.palletsprojects.com/ . Reference for the web application framework.

[9] Dhanush-singh-g, “BDA-Assignment,” https://github.com/Dhanush-singh-g/BDA-Assignment . Project implementation, architecture, deployment documentation and validation record.